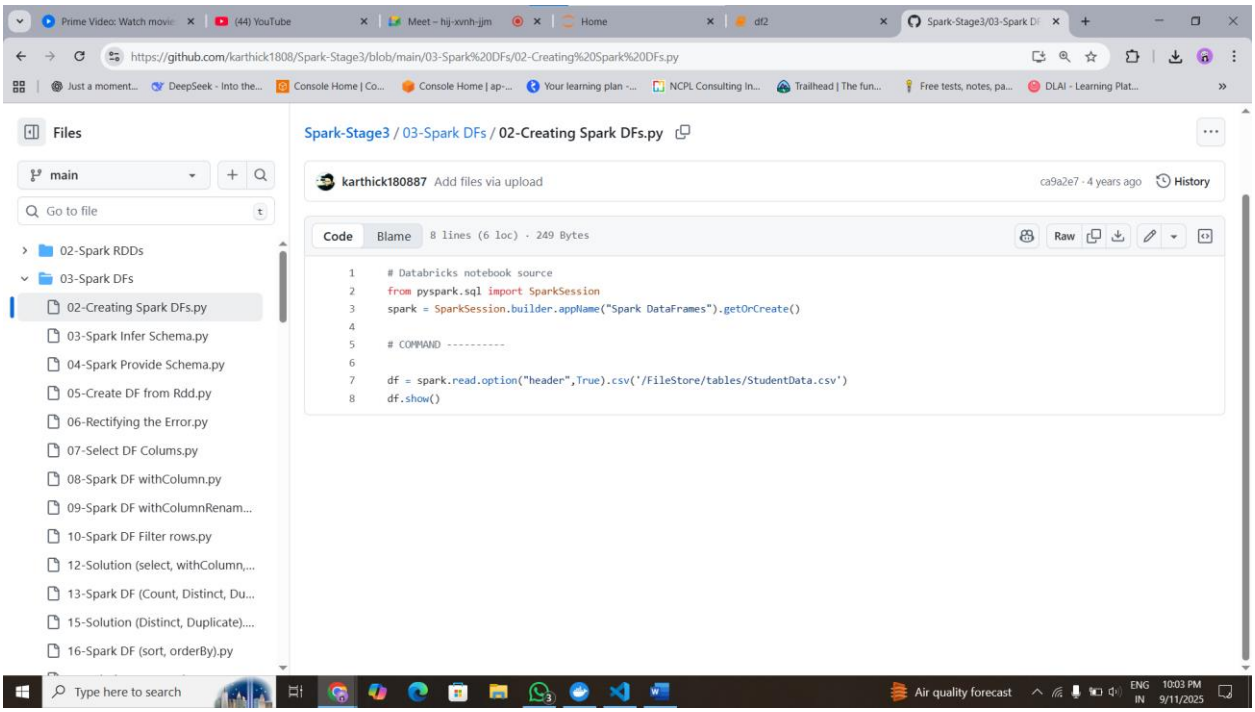
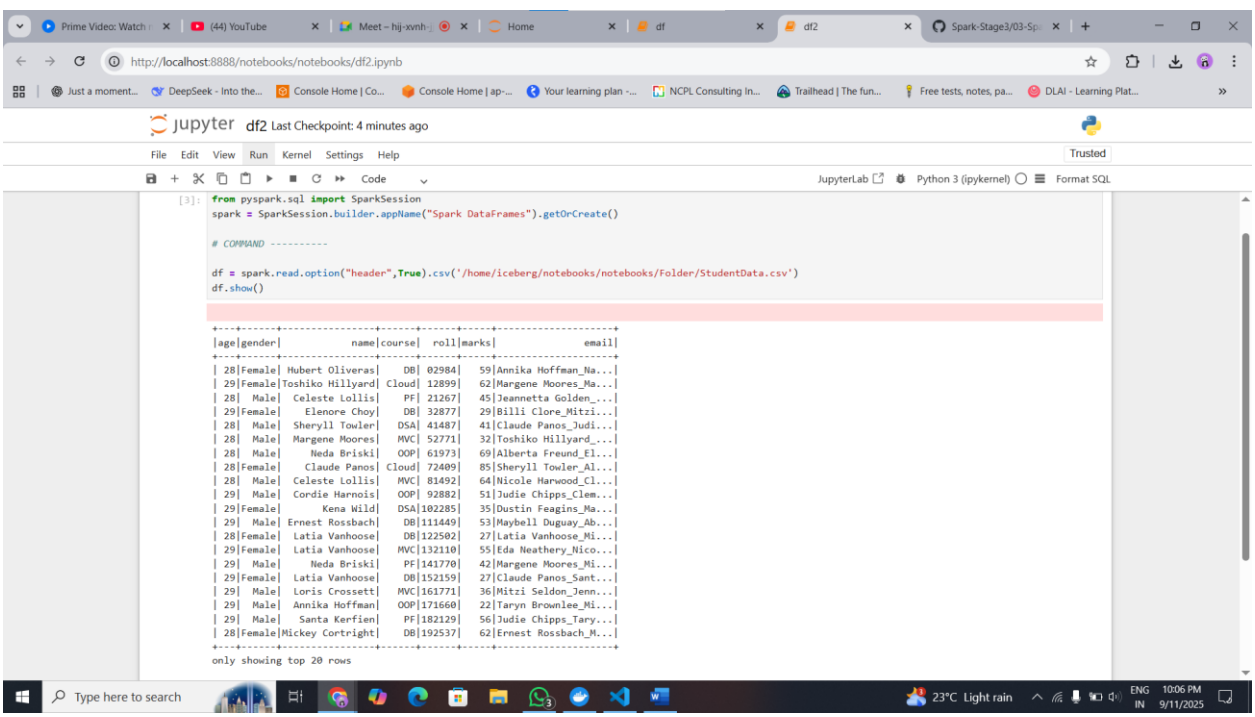
Create a DataFrame from a CSV file: Demonstrated this using spark.read.csv()





Using Schema Inference: (the inferSchema=True) option to automatically detect data types

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()

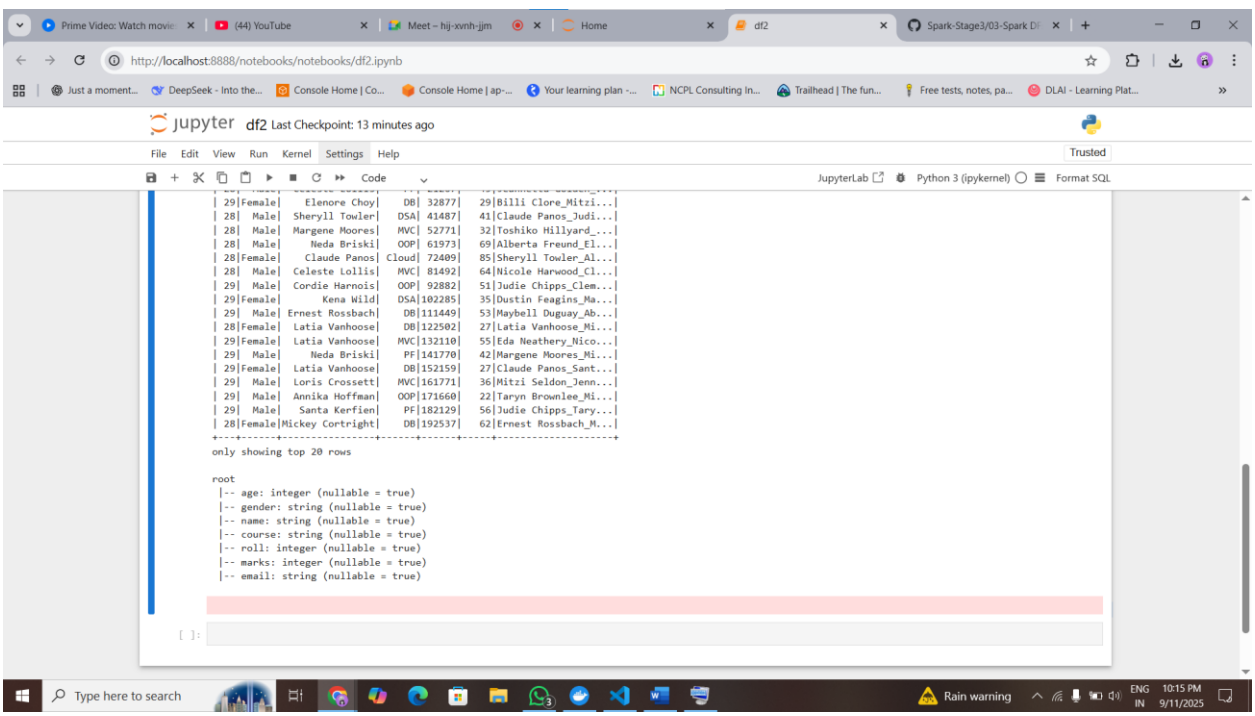
COMMAND ----------

df = spark.read.options(inferSchema='True', header='True', delimiter=',').csv('/FileStore/tables/StudentData.csv')

df.show()

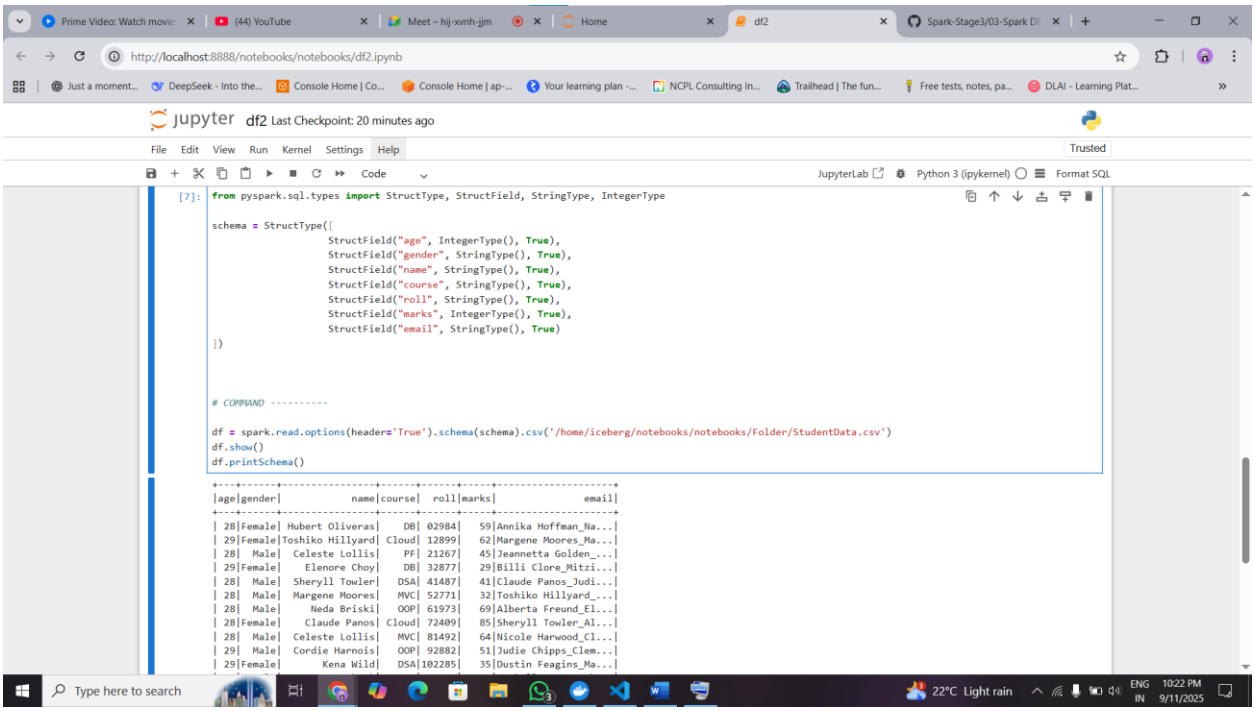
df.printSchema()

```
[6]: from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()

# COMMAND ----------

df = spark.read.options(inferSchema='True', header='True', delimiter=',').csv('/home/iceberg/notebooks/notebooks/Folder/StudentData.csv')
df.show()
df.printSchema()
```



Define and Apply a Custom Schema: How to import StructType and StructField to define a custom schema and apply it when reading a CSV

```
# Databricks notebook source
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()
# COMMAND ----------
df = spark.read.options(inferSchema='True', header='True', delimiter=',').csv('/FileStore/tables/StudentData.csv')
df.show()
df.printSchema()
# COMMAND ----------
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
schema = StructType([
        StructField("age", IntegerType(), True),
        StructField("gender", StringType(), True),
        StructField("name", StringType(), True),
        StructField("course", StringType(), True),
        StructField("roll", StringType(), True),
        StructField("marks", IntegerType(), True),
        StructField("email", StringType(), True)
])
# COMMAND ----------
df = spark.read.options(header='True').schema(schema).csv('/FileStore/tables/StudentData.csv')
df.show()
df.printSchema()
# COMMAND ----------
```



Register a DataFrame as a Temp View and Query it with SQL: Registering a DataFrame as a view and running a spark.sql() query on it.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit, udf
spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()
# COMMAND ----------
df = spark.read.options(header='True', inferSchema='True').csv('/FileStore/tables/StudentData.csv')
df.show()
# COMMAND ----------
df.createOrReplaceTempView("Student")
# COMMAND ----------
spark.sql("select course, gender, count(*) from Student group by course, gender").show()
df.groupBy("course", "gender").count().show()
```

http://localhost:8888/notebooks/notebooks/df2.ipynb

jupyter df2 Last Checkpoint: 25 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel) Format SQL

[8]:
```python
# Databricks notebook source
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit, udf
spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()

# COMMAND ----------

df = spark.read.options(header='True', inferSchema='True').csv('/home/iceberg/notebooks/notebooks/Folder/StudentData.csv')
df.show()

# COMMAND ----------

df.createOrReplaceTempView("Student")

# COMMAND ----------

spark.sql("select course, gender, count(*) from Student group by course, gender").show()
df.groupBy("course", "gender").count().show()
```

```
+---+------+---------------+------+------+-----+--------------------+
|age|gender|           name|course|  roll|marks|               email|
+---+------+---------------+------+------+-----+--------------------+
| 28|Female| Hubert Oliveras|    DB|  2984|   59|Annika Hoffman_Na...|
| 29|Female|Toshiko Hillyard| Cloud| 12899|   62|Margene Moores_Ma...|
| 28|  Male|  Celeste Lollis|    PF| 21267|   45|Jeannetta Golden_...|
| 29|Female|    Elenore Choy|    DB| 32877|   29|Billi Clore_Mitzi...|
| 28|  Male|  Sheryll Towler|   DSA| 41487|   41|Claude Panos_Judi...|
| 28|  Male|  Margene Moores|   MVC| 52771|   32|Toshiko Hillyard_...|
| 28|  Male|     Neda Briski|   OOP| 61973|   69|Alberta Freund_El...|
| 28|Female|    Claude Panos| Cloud| 72409|   85|Sheryll Towler_Al...|
| 28|  Male|  Celeste Lollis|   MVC| 81492|   64|Nicole Harwood_Cl...|
| 29|  Male|  Cordie Harnois|   OOP| 92882|   51|Judie Chipps_Clem...|
| 29|Female|        Kena Wild|   DSA|102285|   35|Dustin Feagins_Ma...|
```

Type here to search 22°C Light rain ENG IN 10:27 PM 9/11/2025

http://localhost:8888/notebooks/notebooks/df2.ipynb

jupyter df2 Last Checkpoint: 25 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel) Format SQL

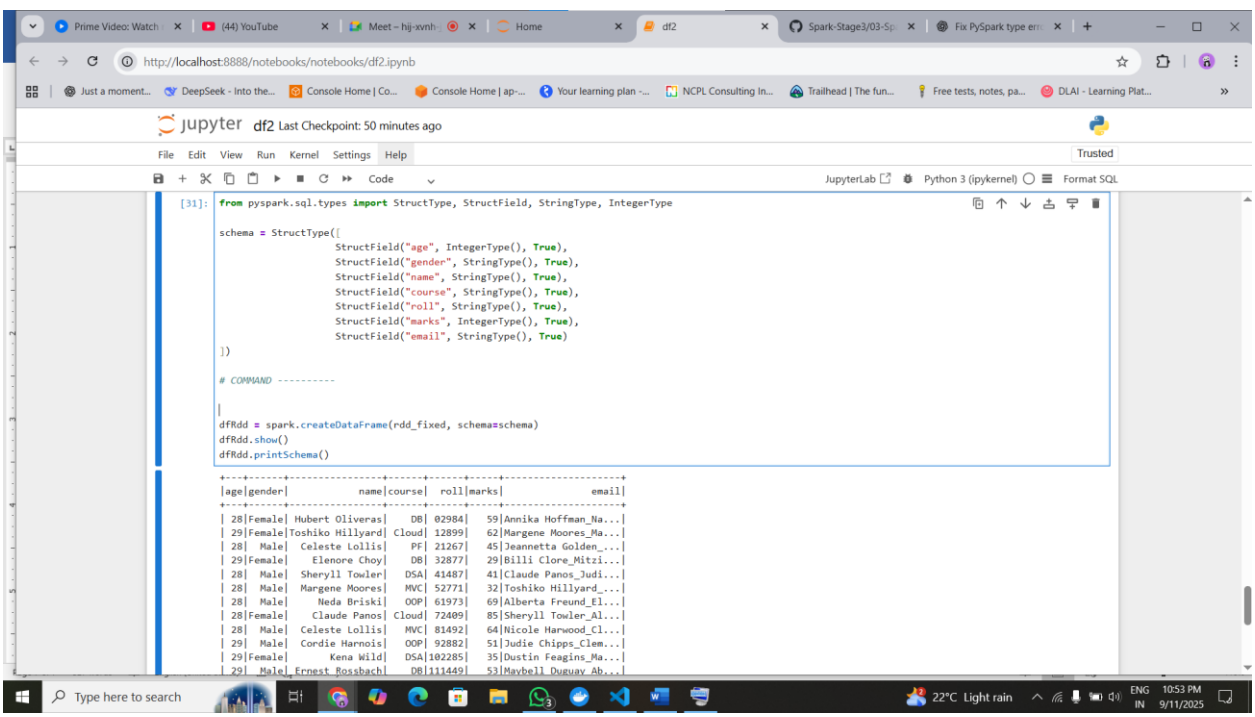
```
+------+------+--------+
|course|gender|count(1)|
+------+------+--------+
|   OOP|  Male|      70|
|    DB|  Male|      82|
| Cloud|Female|     106|
|   MVC|  Male|      86|
|   DSA|Female|      98|
|    PF|  Male|      97|
|   MVC|Female|      71|
| Cloud|  Male|      86|
|    PF|Female|      69|
|   DSA|  Male|      78|
|    DB|Female|      75|
|   OOP|Female|      82|
+------+------+--------+

+------+------+-----+
|course|gender|count|
+------+------+-----+
|   OOP|  Male|   70|
|    DB|  Male|   82|
| Cloud|Female|  106|
|   MVC|  Male|   86|
|   DSA|Female|   98|
|    PF|  Male|   97|
|   MVC|Female|   71|
| Cloud|  Male|   86|
|    PF|Female|   69|
|   DSA|  Male|   78|
|    DB|Female|   75|
|   OOP|Female|   82|
+------+------+-----+
```

Type here to search 22°C Light rain ENG IN 10:28 PM 9/11/2025

Convert an RDD to a DataFrame: Process of reading unstructured data into an RDD, cleaning it (e.g., header removal with filter), and converting it to a structured DataFrame using a defined schema.

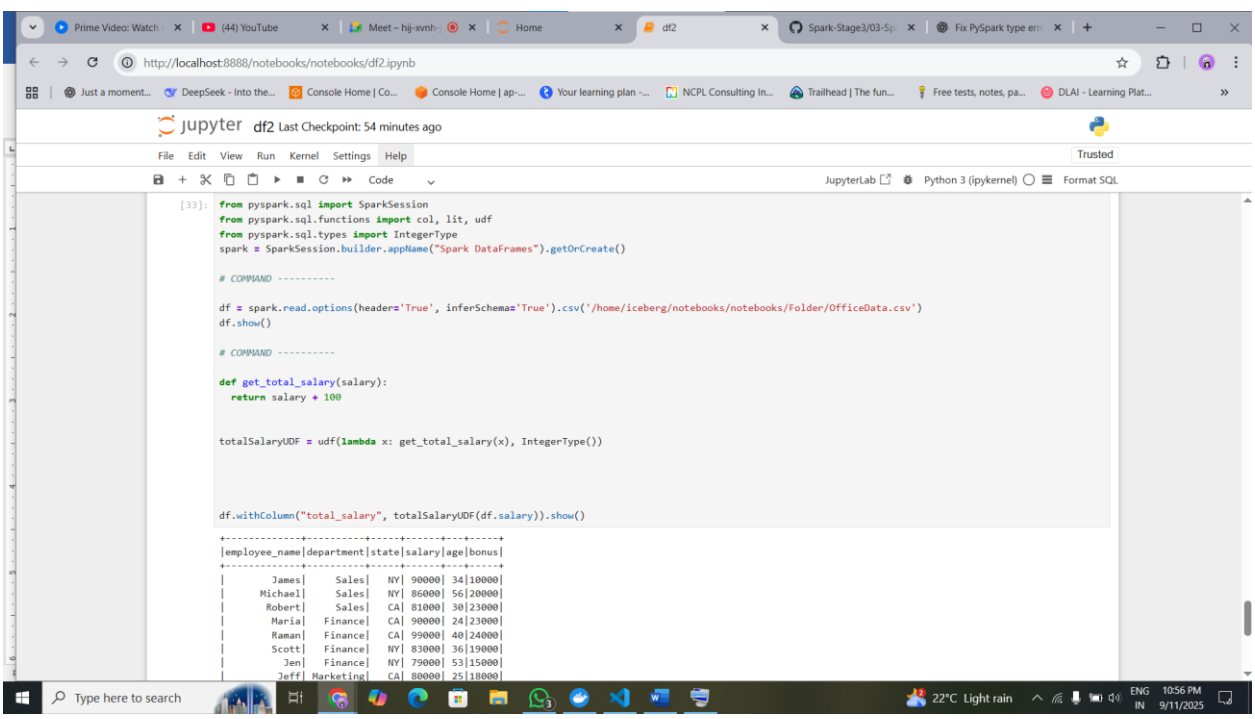
```
# Databricks notebook source
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()

# COMMAND ----------

from pyspark import SparkConf, SparkContext
conf = SparkConf().setAppName("RDD")
sc = SparkContext.getOrCreate(conf=conf)

rdd = sc.textFile('/FileStore/tables/StudentData.csv')
headers = rdd.first()
rdd = rdd.filter(lambda x: x != headers).map(lambda x: x.split(','))

# COMMAND ----------

columns = headers.split(',')
dfRdd = rdd.toDF(columns)
# dfRdd.show()
dfRdd.printSchema()

# COMMAND ----------

from pyspark.sql.types import StructType, StructField, StringType, IntegerType

schema = StructType([
        StructField("age", IntegerType(), True),
        StructField("gender", StringType(), True),
```

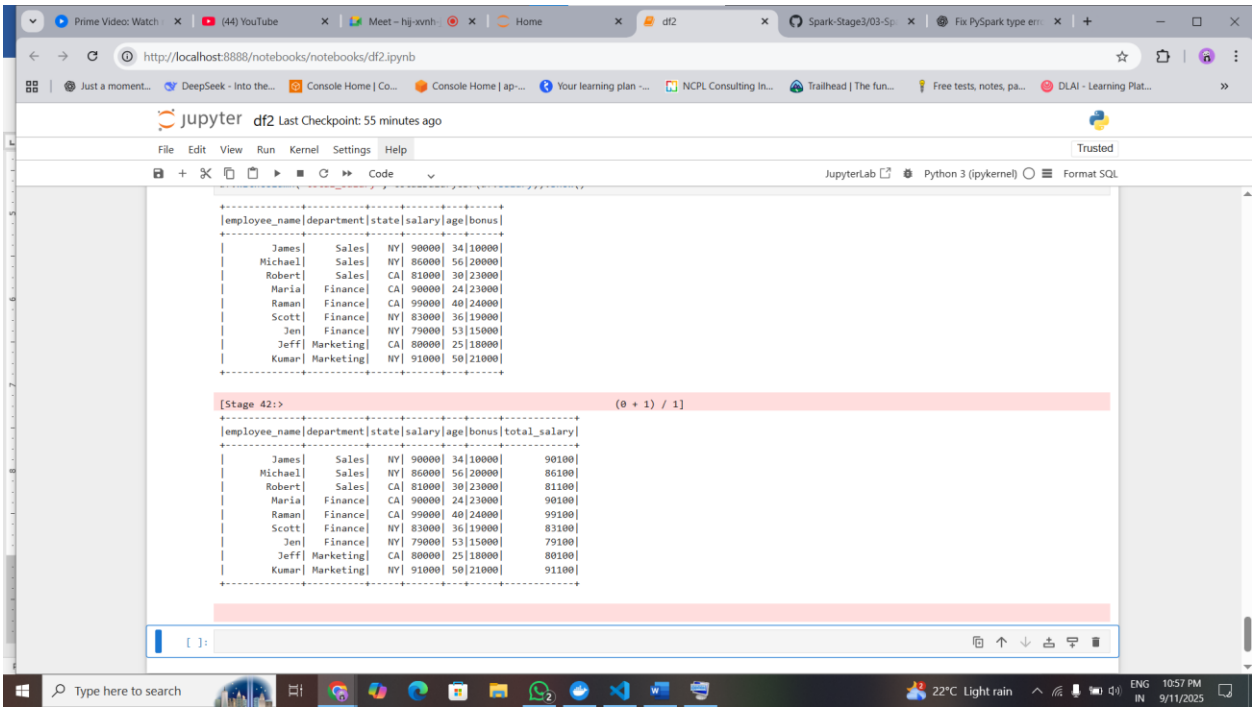
```
            StructField("name", StringType(), True),
            StructField("course", StringType(), True),
            StructField("roll", StringType(), True),
            StructField("marks", IntegerType(), True),
            StructField("email", StringType(), True)
])

# COMMAND ----------


dfRdd = spark.createDataFrame(rdd_fixed, schema=schema)
dfRdd.show()
dfRdd.printSchema()
```



Create and Apply a User-Defined Function (UDF): Implement a UDF for a specific business case (state-based salary bonus) and registered it to use on a DataFrame.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit, udf
from pyspark.sql.types import IntegerType
spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()
# COMMAND ----------
df = spark.read.options(header='True', inferSchema='True').csv('/FileStore/tables/OfficeData.csv')
df.show()
# COMMAND ----------
def get_total_salary(salary):
  return salary + 100
totalSalaryUDF = udf(lambda x: get_total_salary(x), IntegerType())
df.withColumn("total_salary", totalSalaryUDF(df.salary)).show()
```

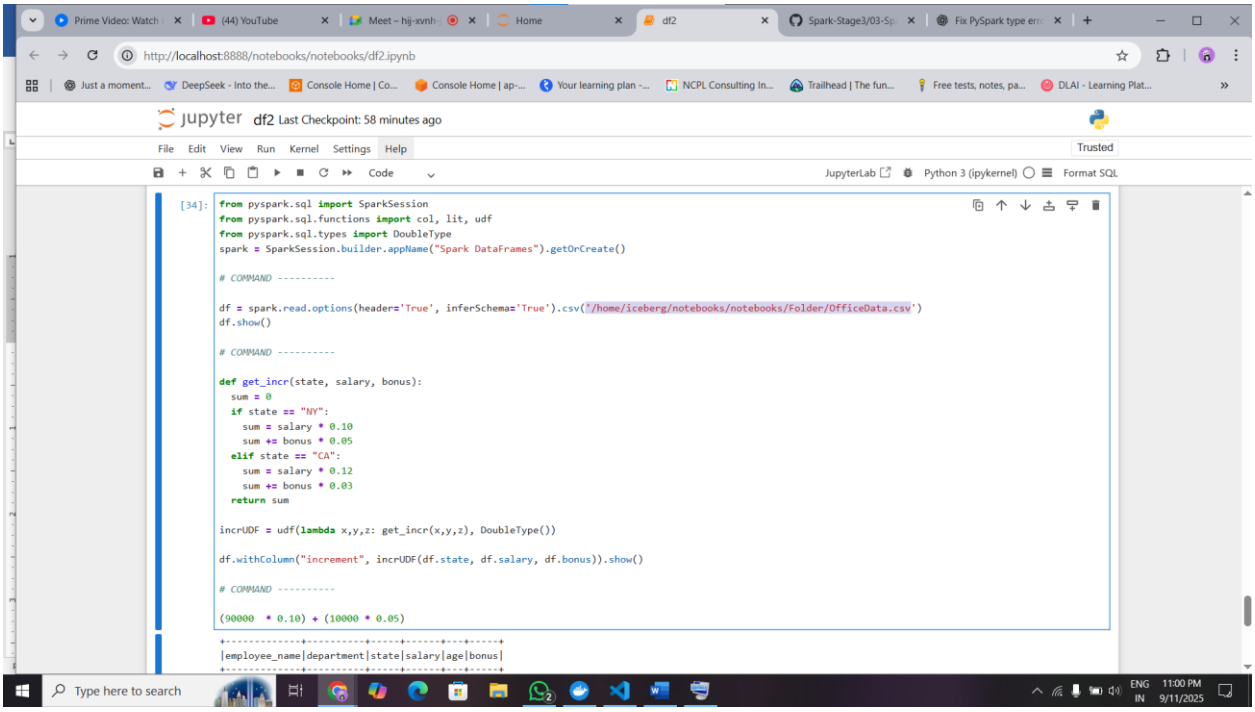


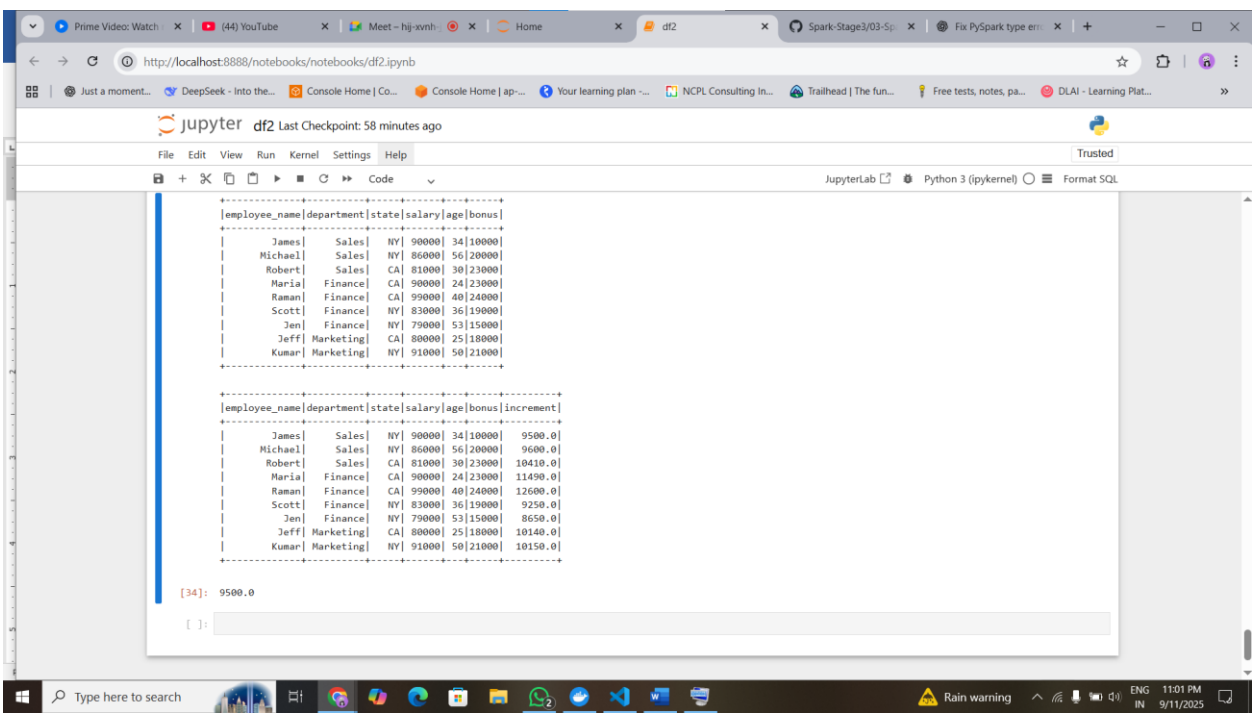


Another types of query:

```
 from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit, udf
from pyspark.sql.types import DoubleType
spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()
# COMMAND ----------
df = spark.read.options(header='True', inferSchema='True').csv('/FileStore/tables/OfficeData.csv')
df.show()
# COMMAND ----------
def get_incr(state, salary, bonus):
  sum = 0
  if state == "NY":
    sum = salary * 0.10
    sum += bonus * 0.05
  elif state == "CA":
    sum = salary * 0.12
    sum += bonus * 0.03
```

```
  return sum
incrUDF = udf(lambda x,y,z: get_incr(x,y,z), DoubleType())
df.withColumn("increment", incrUDF(df.state, df.salary, df.bonus)).show()
# COMMAND ----------
(90000  * 0.10) + (10000 * 0.05)
```





EXPLORE:

DataFrame Creation and Schema Management

Practice creating DataFrames with both inferred and explicit schemas.

* Steps:

1. Download a sample CSV file (e.g., from Kaggle or create one with columns like name, age, city, salary).
2. Read the file into a DataFrame with inferSchema=True. Use printSchema() to see what Spark inferred.
3. Read the same file again, but this time without schema inference (so all columns become strings). Use printSchema() to confirm.
4. Define a custom schema for the DataFrame. Intentionally change one data type (e.g., define age as StringType instead of IntegerType). Apply this schema when reading the file and print the result.

1. Create / Download a Sample CSV

Let's make a simple CSV file called **people.csv**:

name,age,city,salary

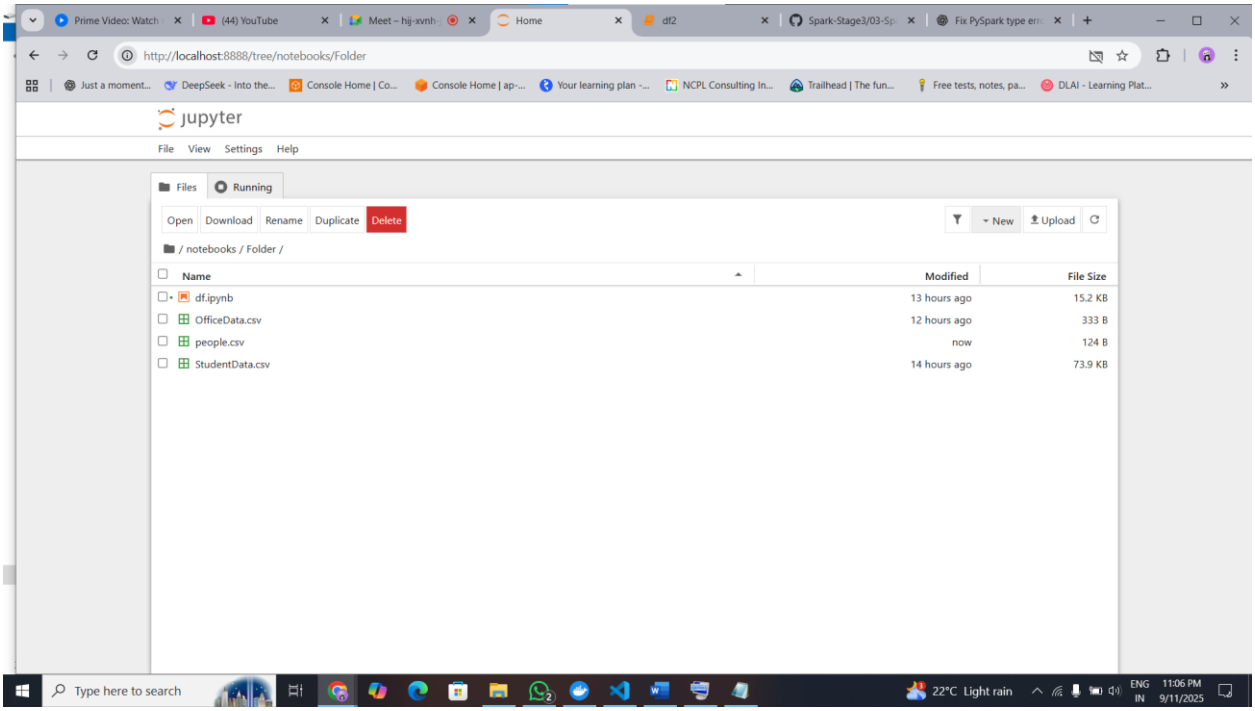
Alice,30,New York,85000

Bob,25,San Francisco,72000

Charlie,28,Chicago,64000

Diana,35,Boston,91000

Save this file in a local path (say /path/to/people.csv) or in HDFS/S3 if you're working in cluster mode.



2. Read with inferSchema=True

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()

COMMAND ----------

df = spark.read.options(inferSchema='True', header='True',

delimiter=',').csv('/home/iceberg/notebooks/notebooks/Folder/people.csv')

df.show()

df.printSchema()

Expected schema:

[36]:
```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()

# COMMAND ----------

df = spark.read.options(inferSchema='True', header='True', delimiter=',').csv('/home/iceberg/notebooks/notebooks/Folder/people.csv')
df.show()
df.printSchema()
```

```
+-------+---+-------------+------+
|   name|age|         city|salary|
+-------+---+-------------+------+
|  Alice| 30|     New York| 85000|
|    Bob| 25|San Francisco| 72000|
|Charlie| 28|      Chicago| 64000|
|  Diana| 35|       Boston| 91000|
+-------+---+-------------+------+

root
 |-- name: string (nullable = true)
 |-- age: integer (nullable = true)
 |-- city: string (nullable = true)
 |-- salary: integer (nullable = true)
```

root

|-- name: string (nullable = true)

|-- age: integer (nullable = true)

|-- city: string (nullable = true)

|-- salary: integer (nullable = true)

---

3. Read without schema inference (all strings)

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()

COMMAND ----------

df = spark.read.options(header='True', delimiter=',').csv('/home/iceberg/notebooks/notebooks/Folder/people.csv')

df.show()

df.printSchema()

Expected schema:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Spark DataFrames").getOrCreate()

# COMMAND ----------

df = spark.read.options( header='True', delimiter=',').csv('/home/iceberg/notebooks/notebooks/Folder/people.csv')
df.show()
df.printSchema()
```

```
+-------+---+-------------+------+
|   name|age|         city|salary|
+-------+---+-------------+------+
|  Alice| 30|     New York| 85000|
|    Bob| 25|San Francisco| 72000|
|Charlie| 28|      Chicago| 64000|
|  Diana| 35|       Boston| 91000|
+-------+---+-------------+------+

root
 |-- name: string (nullable = true)
 |-- age: string (nullable = true)
 |-- city: string (nullable = true)
 |-- salary: string (nullable = true)
```

root

|-- name: string (nullable = true)

|-- age: string (nullable = true)

|-- city: string (nullable = true)

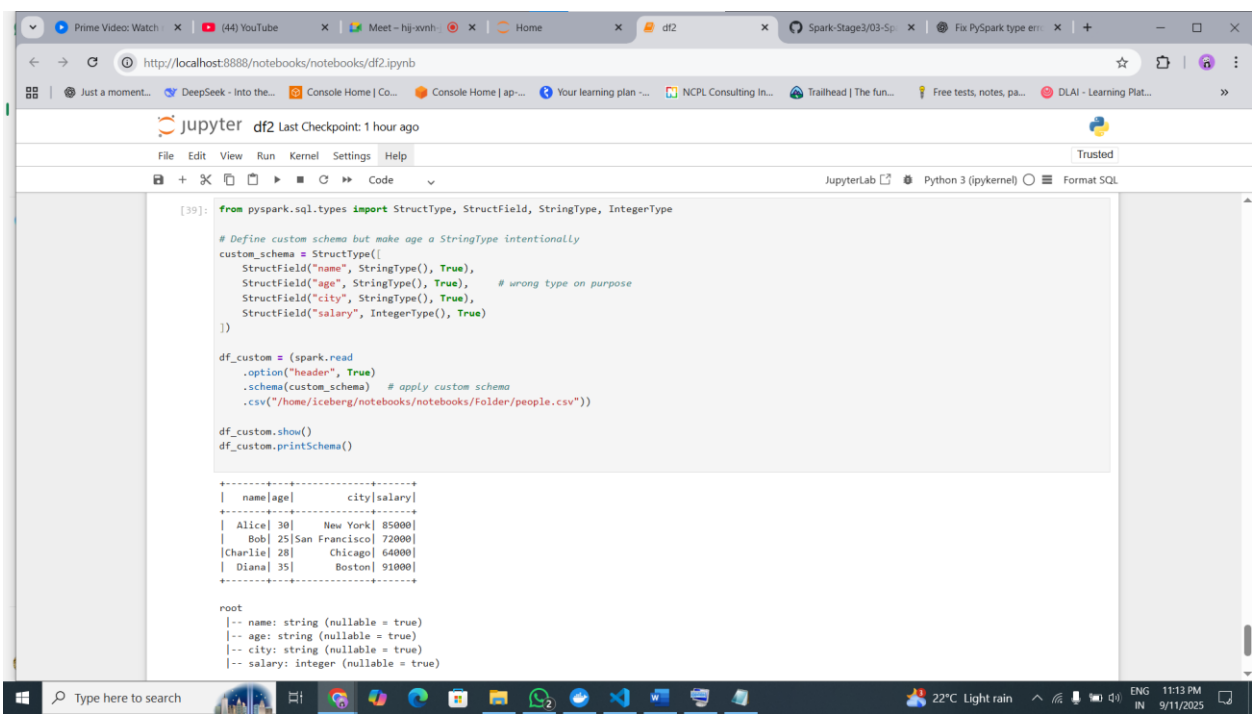
|-- salary: string (nullable = true)

---

4. Apply a Custom Schema (with intentional mistake)

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

# Define custom schema but make age a StringType intentionally
custom_schema = StructType([
    StructField("name", StringType(), True),
    StructField("age", StringType(), True),     # wrong type on purpose
    StructField("city", StringType(), True),
    StructField("salary", IntegerType(), True)
])

df_custom = (spark.read
    .option("header", True)
    .schema(custom_schema)   # apply custom schema
    .csv("/path/to/people.csv"))

df_custom.show()
df_custom.printSchema()
```

Expected schema:



```
root
 |-- name: string (nullable = true)
 |-- age: string (nullable = true)      Spark now treats age as string
 |-- city: string (nullable = true)
 |-- salary: integer (nullable = true)
```

With this exercise you'll **see the differences clearly**:

- InferSchema automatically converts numbers → IntegerType.
- No inference = all strings.
- Custom schema overrides everything (even if it's wrong).

ETL Pipeline: From Unstructured Data to Insight

We'll simulate a real-world ETL process using PySpark. The input is a **messy server log–style file**, and the output is **average salary per department**.

---

1. Input File (Raw Data)

File: people.txt

```
id,name,dept,salary
101,John,D1,50000
102,Jane,D2,60000
bad_record
103,Alex,D1,55000
```

Problems with this data:

- Has a **header row**.
- Contains a **bad record** (bad_record) that does not match schema.

---

2. PySpark ETL Code

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

# ---------------------------------------------------------
# Step 1: Create Spark Session
# ---------------------------------------------------------
spark = SparkSession.builder.appName("ETL-Pipeline").getOrCreate()

# ---------------------------------------------------------
# Step 2: Read raw file as RDD
# ---------------------------------------------------------
rdd_raw = spark.sparkContext.textFile("/path/to/employee_data.txt")

# ---------------------------------------------------------
# Step 3: Clean RDD
#   - Remove header
```

```
#   - Filter out bad records
# ---------------------------------------------------------
header = rdd_raw.first()
rdd_cleaned = (rdd_raw
    .filter(lambda line: line != header)
    .filter(lambda line: "bad_record" not in line))

# ---------------------------------------------------------
# Step 4: Transform into structured format
# ---------------------------------------------------------
rdd_structured = rdd_cleaned.map(lambda line: line.split(","))

# ---------------------------------------------------------
# Step 5: Define schema & create DataFrame
# ---------------------------------------------------------
schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("dept", StringType(), True),
    StructField("salary", IntegerType(), True)
])

df = spark.createDataFrame(
    rdd_structured.map(lambda x: (int(x[0]), x[1], x[2], int(x[3]))),
    schema
)

print("Cleaned DataFrame")
df.show()
df.printSchema()

# ---------------------------------------------------------
```

```
# Step 6: Analyze - Average salary by department
# ---------------------------------------------------------
result = df.groupBy("dept").avg("salary")

print(" Average Salary by Department")
result.show()

# ---------------------------------------------------------
# Step 7: (Optional) Save result to CSV
# ---------------------------------------------------------
result.write.mode("overwrite").csv("/path/to/output/avg_salary_by_dept")

spark.stop()
```

---

3. Expected Output

Cleaned DataFrame

```
+---+-----+----+------+
| id| name|dept|salary|
+---+-----+----+------+
|101| John|  D1| 50000|
|102| Jane|  D2| 60000|
|103| Alex|  D1| 55000|
+---+-----+----+------+
```

Schema

```
root
 |-- id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- dept: string (nullable = true)
 |-- salary: integer (nullable = true)
```

Average Salary by Department

```
+----+------------+
|dept|avg(salary)|
```

```
+----+-----------+
| D1|   52500.0|
| D2|   60000.0|
+----+-----------+
```

With this one script, you've completed a **mini ETL job**:

- **Extract** → from raw unstructured text.
- **Transform** → remove header, filter corrupt data, structure records.
- **Load** → into DataFrame.
- **Analyze** → calculate average salary per department.
- **Optional** → save results back to storage.

*Similar type of code or program is executed above in Karthick's sir work.

SQL vs DataFrame API in Spark

We'll reuse the **people.csv** dataset from Task 1:

```
name,age,city,salary
Alice,30,New York,85000
Bob,25,San Francisco,72000
Charlie,28,Chicago,64000
Diana,35,Boston,91000
```

---

1. Load DataFrame

```
df = (spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv("/path/to/people.csv"))

df.show()
df.printSchema()
```

Schema:

```
root
 |-- name: string (nullable = true)
 |-- age: integer (nullable = true)
 |-- city: string (nullable = true)
 |-- salary: integer (nullable = true)
```

---

2. Register as Temp View

```
df.createOrReplaceTempView("people_view")
```

---

3. Query with SQL

```
sql_result = spark.sql("""
    SELECT name, age, city
    FROM people_view
    WHERE age > 30
""")
```

```
sql_result.show()
```

Output:

```
+-----+---+------+
| name|age|  city|
+-----+---+------+
|Diana| 35|Boston|
+-----+---+------+
```

---

4. Query with DataFrame API

```
df_result = (df
    .select("name", "age", "city")
    .filter(df.age > 30))

df_result.show()
```

Output (identical to SQL):

```
+-----+---+------+
| name|age|  city|
+-----+---+------+
|Diana| 35|Boston|
+-----+---+------+
```

---

5. Compare Execution Plans

```
print("SQL Query Execution Plan:")
sql_result.explain()

print("\nDataFrame API Execution Plan:")
df_result.explain()
```

*Similar type of code or program is executed above in Karthick’s sir work.

Implementing Complex Business Logic with UDFs

1. Sample Employee Data

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder.appName("UDF-Example").getOrCreate()

# Sample data
data = [
    (1, "Alice", "CA", 85000),
    (2, "Bob", "NY", 72000),
    (3, "Charlie", "TX", 64000),
    (4, "Diana", "CA", 91000),
    (5, "Ethan", "FL", 58000)
]

schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("state", StringType(), True),
    StructField("salary", IntegerType(), True)
])

df = spark.createDataFrame(data, schema)
df.show()
```

Output:

```
+---+-------+-----+------+
| id|   name|state|salary|
+---+-------+-----+------+
|  1|  Alice|   CA| 85000|
|  2|    Bob|   NY| 72000|
|  3|Charlie|   TX| 64000|
```

```
|  4|  Diana|   CA| 91000|
|  5|  Ethan|   FL| 58000|
+---+-------+-----+------+
```

---

2. Define the UDF

```
from pyspark.sql.functions import udf
from pyspark.sql.types import DoubleType

# Python function for bonus logic
def calculate_bonus(state, salary):
    if state == "CA":
        return salary * 0.15
    elif state == "NY":
        return salary * 0.10
    else:
        return salary * 0.05

# Register UDF
bonus_udf = udf(calculate_bonus, DoubleType())
```

---

3. Apply UDF to Create bonus Column

```
df_with_bonus = df.withColumn("bonus", bonus_udf(df.state, df.salary))
df_with_bonus.show()
```

Output:

```
+---+-------+-----+------+-------+
| id|   name|state|salary|  bonus|
+---+-------+-----+------+-------+
|  1|  Alice|   CA| 85000|12750.0|
|  2|    Bob|   NY| 72000| 7200.0|
|  3|Charlie|   TX| 64000| 3200.0|
|  4|  Diana|   CA| 91000|13650.0|
|  5|  Ethan|   FL| 58000| 2900.0|
```

```
+---+-------+-----+------+-------+
```

---

4. Add total_comp Column

```
from pyspark.sql.functions import col

df_final_udf = df_with_bonus.withColumn("total_comp", col("salary") + col("bonus"))
df_final_udf.show()
```

Output:

```
+---+-------+-----+------+-------+----------+
| id|   name|state|salary|  bonus|total_comp|
+---+-------+-----+------+-------+----------+
|  1|  Alice|   CA| 85000|12750.0|   97750.0|
|  2|    Bob|   NY| 72000| 7200.0|   79200.0|
|  3|Charlie|   TX| 64000| 3200.0|   67200.0|
|  4|  Diana|   CA| 91000|13650.0|  104650.0|
|  5|  Ethan|   FL| 58000| 2900.0|   60900.0|
+---+-------+-----+------+-------+----------+
```

---

5. (Challenge) Without UDF → Using Built-in Functions

```
from pyspark.sql.functions import when

df_final_builtin = (df
    .withColumn("bonus",
        when(col("state") == "CA", col("salary") * 0.15)
        .when(col("state") == "NY", col("salary") * 0.10)
        .otherwise(col("salary") * 0.05)
    )
    .withColumn("total_comp", col("salary") + col("bonus"))
)

df_final_builtin.show()
```

Output (same as UDF):

```
+---+-------+-----+------+-------+----------+
| id|   name|state|salary|  bonus|total_comp|
+---+-------+-----+------+-------+----------+
|  1|  Alice|   CA| 85000|12750.0|   97750.0|
|  2|    Bob|   NY| 72000| 7200.0|   79200.0|
|  3|Charlie|   TX| 64000| 3200.0|   67200.0|
|  4|  Diana|   CA| 91000|13650.0|  104650.0|
|  5|  Ethan|   FL| 58000| 2900.0|   60900.0|
+---+-------+-----+------+-------+----------+
```

UDF vs Built-in Functions

- **UDF Pros**:
  - Flexible (any Python logic).
  - Can handle very custom rules.
- **UDF Cons**:
  - Runs row-by-row in Python (slower).
  - Breaks Spark's optimization (Catalyst can't peek inside).
- **Built-in Functions Pros**:
  - Faster (vectorized, runs in JVM).
  - Catalyst optimizer can optimize queries.
  - More readable for simple logic.

Rule of Thumb: **Use built-in functions whenever possible; UDF only when business logic is too complex.**

*Similar type of code or program is executed above in Karthick's sir work.